# Learned deferral for sanctions entity resolution

## Classical matchers, LLMs and human review

Harsh Chauhan, EP23BT012

Pattern Recognition and Machine Learning, Course Project
Indian Institute of Technology Dharwad

November 2026

# Outline


1. Motivation
2. Data and features
3. Methods, theory and implementation
4. Results
5. Robustness
6. Conclusions

# Problem: Entity resolution in sanctions screening

- Entity resolution asks whether two records from different sources describe the same real-world person, company or vessel.
- Banks, payment companies, exchanges and investigative journalists screen customer and counterparty names against sanctions and watch lists every day. OpenSanctions alone merges records from 293 sources into about a million entities.
- Both mistakes are costly. A missed match lets a sanctioned entity through, which is a legal and reputational problem. A false match blocks a legitimate customer and sends the case to an analyst.
- Analyst review is the expensive part. The practical question is not only "is this a match" but "which pairs deserve a human look".

# Why it is hard

- The same person appears under different transliterations, name orders and scripts, and often under aliases.
- Sources record different subsets of fields. One list gives a birth date and a position, another a birth place and an identifier, a third only a name.
- Different people share names. Namesakes can only be separated by dates, places and identifiers, and those are often missing.
- Data-entry errors are common, so a conflicting date or identifier is not proof of a different entity.

**Record A (sanctions list 1)**

```
type:  Person
name:  Vitaliy Anatolyevich Markelov
other names:  MARKELOV, Vitaliy; Markelov Vitaly
Anatolievich
birth date:  1963-08-05   nationality:  ru
position:  Deputy Chairman, Gazprom
```

**Record B (sanctions list 2)**

```
type:  Person
name:  Witalij Anatoljewicz Markielow
birth date:  1963-08-05   citizenship:  ru
birth place:  Mordovia   wikidata:  Q114246932
```

The analysts labelled this pair **same entity**.

# The OpenSanctions Pairs benchmark

- OpenSanctions released the first large public benchmark for this task in 2026. It has 755,540 record pairs judged by their analysts, drawn from over a million entities, 293 sources and 45 jurisdictions.
- Their paper reports 91.3 F1 for the production scorer, a small regression over a few rule features, and 94 to 99 F1 for LLMs from Llama 8B up to GPT-4o.
- They name uncertainty-aware review as the open problem. Nobody has studied which pairs should go to a human and what that review buys.
- Nothing stronger than the small regression scorer has been trained on these 755k labels, although labelled pairs are exactly what supervised record linkage needs.

# The gap and our three questions

- **Question 1.** Can an ordinary classifier, trained on these analyst labels, beat the LLMs? How many labelled pairs does it need?
- **Question 2.** If a human can only review a few percent of the pairs, which ones should go to them, and how much does that review buy?
- **Question 3.** Is an LLM useful as that reviewer or does it fail on the same pairs the classifier fails on?

# Data

- Public file `pairs-20251209.json.gz`. Each line is two records and an analyst's decision, same entity or not. Records carry names and aliases, dates, nationality, identifiers and addresses.
- Records also carry bookkeeping fields about their history in the OpenSanctions database (`datasets`, `referents`, `first_seen`, `target`). Some of them encode the answer, so we delete them before computing anything.
- 37% of the lines are not entity pairs but relationship records (Occupancy, Ownership) copied identically on both sides and always labelled same. They make any model look perfect, and our first run showed a meaningless 99.4 F1. We keep only the ten real entity types.
- After removing duplicate pairs there are 264k same and 171k different. We draw a fixed random sample of 60k (39k same, 21k different) and split it into 36k train, 6k validation and 18k test.

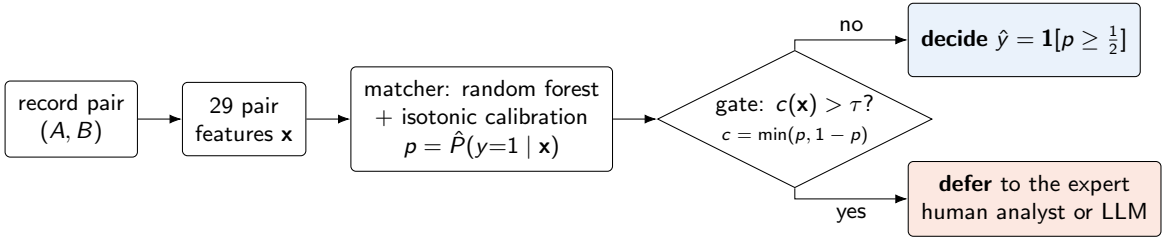
# Features

- A classifier takes a fixed list of numbers, not two JSON records. So for every pair we compute 29 numbers, each answering one small question about how the two records relate. That list is what the classifiers learn from.
- **Names** are the most important group. A record can have several names (the main name and its aliases), so we compare every name of A with every name of B and keep the best score.
- Jaro-Winkler and Levenshtein ratio measure how close two spellings are, so `Mohammed` and `Muhammad` score high. Token set ratio and token Jaccard compare the words while ignoring their order, so `Naqdi Mohammad Reza` and `Mohammad Reza Naqdi` count as equal.
- We also record whether any two names are exactly equal after lowercasing and stripping accents, whether the first words and the last words agree, and whether the two names use different scripts (Latin against Cyrillic or Arabic), because then the spelling scores mean nothing.

# Features

- **Type and dates.** Whether both records are the same kind of entity (a Person against a Company is an instant no), whether both have a birth or registration date, whether the full dates agree or only the years, how many years apart they are, and whether a date is missing on one side.
- **Countries, gender, identifiers.** Whether the records share a nationality or country, or both list countries and share none. Whether one is male and the other female. Identifiers are passport, tax and registration numbers, and we record whether both sides have one, whether any matches, and whether both have one and none match. Same name and birth year but different passport numbers means two different people.
- **Other fields.** Shared address, position, birthplace, and OpenSanctions topic tag such as sanction or politician.
- **Five dropped features** counted the names, aliases and properties on each record. They predict the label well but for a bad reason. An entity on many sanction lists has been merged many times, so it has many aliases and also appears in many same pairs. A new record arriving at a bank has one name, so the feature means nothing in practice. We removed them.

# The pipeline




- The matcher turns the features into a calibrated probability $p$ that the pair is the same entity.
- The gate scores each pair by the matcher's uncertainty, $c(\mathbf{x}) = \min(p, 1-p)$, which is the chance that the matcher's answer is wrong. The threshold $\tau$ comes from the review budget. With 5% to spend, $\tau$ is set so that exactly the 5% most uncertain pairs lie above it.
- Pairs below $\tau$ are decided by the matcher, pairs above it go to the expert. Everything upstream is shared, so the human and the LLMs are compared on exactly the same pairs.

# The seven classifiers

- All seven see the same 29 features (standardised where the model needs it) and the same 36k training pairs, except the SVM, which is fitted on a 12k subsample for speed. The comparison is about the model and not the input. Class weights handle the 65 to 35 class imbalance.
- **Logistic regression.** A linear score through the sigmoid, $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, trained by minimising the class weighted log loss plus an L2 penalty $\frac{1}{2}\|\mathbf{w}\|^2$ that shrinks the weights. $C = 1$ sets the trade off between the two terms. The boundary is a hyperplane, so this is the linear baseline.
- **SVM with RBF kernel.** The boundary with the widest margin, min $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$, where $C = 3$ prices the margin violations $\xi_i$. The Gaussian kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ makes the boundary curved, with $\gamma$ at its default scale value. Probabilities come from Platt scaling, a sigmoid fitted to the signed distance.
- ***k*-nearest neighbours.** No training. The $k = 25$ closest training pairs vote, weighted by inverse distance, $p = \sum_i y_i/d_i \,/\, \sum_i 1/d_i$. It works here because the features are already similarity scores, so nearby in feature space means a similar pair.

# The seven classifiers

- **Decision tree.** A sequence of yes or no questions on single features. Each split is chosen greedily to reduce the Gini impurity $G = 2q(1-q)$ of the child nodes, where $q$ is the fraction of same pairs in a node. Depth at most 8, at least 20 pairs per leaf, and the leaf fraction is the probability. One tree is crude, but it is the building block of the next three.
- **Random forest.** 400 deep trees, each grown on a bootstrap sample with a random subset of features at every split, averaged. It is our matcher and has its own slide after the protocol.
- **AdaBoost.** Trees are added one at a time. After round $t$ every misclassified pair has its weight multiplied by $e^{\alpha_t}$ with $\alpha_t = \ln \frac{1-\epsilon_t}{\epsilon_t}$, so the next tree concentrates on it, and the final score is $\sum_t \alpha_t h_t(\mathbf{x})$. 300 trees of depth 2, learning rate 0.5.
- **Gradient boosting.** Also sequential, but each new tree is fitted to the gradient of the log loss, the residual $y_i - \sigma(F_{m-1}(\mathbf{x}_i))$, and added with a small step, $F_m = F_{m-1} + \eta\, h_m$. 400 rounds, $\eta = 0.06$, at most 31 leaves per tree.

# How the models were compared

- The seven classifiers were compared by 5-fold cross-validation on the 36k training pairs alone. The forest won on F1 and AUC, so from here on the matcher means the random forest.
- The decision threshold and the calibration were fitted on the 6k validation pairs and then frozen. For the forest the threshold is 0.5.
- The 18k test pairs were scored once per model. We report F1, AUC and the Brier score, and every random choice has a fixed seed so all numbers reproduce exactly.

# The random forest, our matcher

- **Bagging.** Each of the $T = 400$ trees is grown on a bootstrap sample (36k pairs drawn with replacement) down to leaves of at least 2 pairs. One deep tree has low bias and high variance. Averaging many keeps the bias low and removes most of the variance.
- **Random feature subsets.** At every split a tree may only choose among a random subset of the features, here $\sqrt{29} \approx 5$ of them, which makes the trees different from each other. If every tree has variance $\sigma^2$ and any two trees have correlation $\rho$, the variance of their average is

$$\rho\,\sigma^2 + \frac{1-\rho}{T}\,\sigma^2,$$

  so more trees remove the second term and decorrelation shrinks the first.
- **The score.** $s(\mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} h_t(\mathbf{x})$, where $h_t(\mathbf{x})$ is the fraction of same pairs in the leaf of tree $t$ that the pair lands in. This vote fraction is what isotonic calibration later turns into $p$. Balanced subsample weights rebalance the classes inside every bootstrap sample.
- **Why this one.** Best cross validated F1 and AUC of the seven, best Brier score on test, fits in 7 seconds, needs no feature scaling, and scales linearly to the full data set. Its feature importances rank the four name scores first, then exact name match and identifier conflict.

# Probability calibration

- The forest's score $s$ is a vote fraction. If 380 of the 400 trees say same, $s = 0.95$. It ranks pairs almost perfectly (AUC 0.997), but it is not yet a probability. Take all pairs with $s = 0.8$ and the real share of matches among them might be 0.7 or 0.9.
- The gate and the Bayes rule treat $p$ as the real probability of a match, so $\min(p, 1-p)$ must be a real chance of being wrong. That only holds if 0.8 means 80%.
- Isotonic regression fits that correction on the 6k validation pairs. Sort them by $s$ and find the step function $g$ that never decreases and matches the labels best,

$$g = \arg \min_{g \text{ non-decreasing}} \sum_{i \in \text{val}} \big(y_i - g(s_i)\big)^2, \qquad p = g(s).$$

  Each step is simply the observed match rate at that score level.
- It is fitted on validation only, never on test, and it only relabels the scale, so the order of the pairs and the AUC do not change.

# Bayes decision theory with a reject option

- For each pair we can say same, say different, or hand it to the expert. Each choice has an expected cost. A wrong same costs $c_{\mathrm{FP}}$, a wrong different costs $c_{\mathrm{FN}}$, a review costs $c_d$.
- We pick the cheapest. Working it out, the pair goes to the expert when $p$ falls in the middle region

$$\frac{c_d}{c_{\mathrm{FN}}} \;<\; p \;<\; 1 - \frac{c_d}{c_{\mathrm{FP}}}\,.$$

  With equal costs this is just $\min(p, 1-p) > \tau$, the gate on the pipeline slide.
- If a missed match is the costlier mistake, the region shifts so that more possible matches get reviewed.
- In practice the team can review a fixed share, say 5%, so we send the 5% most uncertain pairs. Varying the share gives the F1 against budget curves in the results.
- Two reference gates for comparison. Random picks the same share of pairs at random, oracle picks exactly the matcher's mistakes.

# Learning to defer, the learned gates

- So far the gate ranks pairs by $\min(p, 1-p)$ and nothing else. It knows how unsure the matcher is, but not why. Two names in different scripts, a missing birth date, or conflicting identifiers might make a pair riskier than $p$ alone admits.
- The natural next question is whether a second model can rank the pairs better by also looking at the 29 features and the raw forest score. This is the learning to defer idea from the literature.
- We train two such gates. The error gate predicts "the matcher is wrong on this pair". The gain gate predicts "the matcher is wrong and the expert will be right", which is the right target once the expert is an LLM that can itself be wrong.
- Both are logistic regression and gradient boosting, trained on 12k training pairs for which we also have LLM answers, kept apart from calibration and test. Every gate is then compared at the same review budgets.
- If a learned gate wins, the features carry information about mistakes that $p$ misses. If not, the calibrated probability already says everything there is to say. The results decide.

# The expert, a human analyst or an LLM

- The gate decides which pairs leave the matcher. Someone has to answer them. We try two experts. The human analyst is simulated by the true label, so that curve is the best any reviewer could do. The LLM expert is run as described below.
- Models. Qwen2.5-7B, Qwen2.5-14B in 4-bit, the 7B again with four worked examples in the prompt, and later Qwen3-32B in 4-bit, zero-shot, with a short chain of thought, and in its thinking mode. All served with vLLM at temperature 0, so every answer is deterministic.
- Prompt. The model is told it is an analyst doing entity resolution, warned that names may be transliterated or abbreviated, that small date or identifier differences are often typing errors, and that different people can share a name. Then it sees the two records as labelled lines, about 417 tokens, and must answer with one word, yes or no. The full prompt is in the backup slides.
- The probability of yes is read from the first-token log probabilities, so the LLM gets a score between 0 and 1 like every other classifier.
- Two roles. As a matcher it decides every pair alone and is scored like the seven classifiers. As the expert its answer replaces the forest's on the deferred pairs only. A 600-pair check (95.9 F1) preceded the 30k-pair runs (95.8).

# Implementation

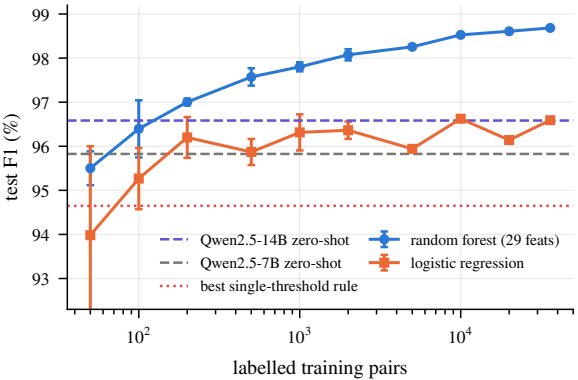
- Python with scikit-learn for the classifiers, calibration and gates, rapidfuzz for the string similarities, vLLM to run the language models.
- One script per stage, in order: sample and clean the data, split, compute the features, train and compare the classifiers, calibrate, score the LLMs, run the gates and deferral curves, analyse the hard core. A notebook runs the stages in sequence and keeps every output and figure.
- Everything except the LLM scoring runs on a CPU in a few minutes. The LLM runs needed one GPU and took between a few minutes and half an hour per model.
- Fixed seeds and a saved split, so every number in the paper and these slides is reproduced by the notebook.

# Matcher results

| classifier | CV F1 | test F1 | F2 | AUC |
|---|---|---|---|---|
| logistic regression | 96.5 | 97.1 | 97.7 | 0.989 |
| decision tree | 98.1 | 98.1 | 98.4 | 0.992 |
| AdaBoost | 98.0 | 98.0 | 98.0 | 0.995 |
| SVM (RBF) | 98.5 | 98.5 | 98.6 | 0.995 |
| kNN | 98.5 | 98.6 | 98.8 | 0.995 |
| gradient boosting | 98.7 | 98.6 | 98.8 | 0.997 |
| **random forest** | **98.7** | **98.7** | **98.9** | **0.997** |
| Qwen2.5-7B zero-shot | | 95.8 | | |
| Qwen2.5-14B zero-shot | | 96.6 | | |
| Qwen3-32B zero-shot | | 96.8 | | |
| best single rule | | 94.6 | 94.4 | |
| production scorer | | 91.3 | | |

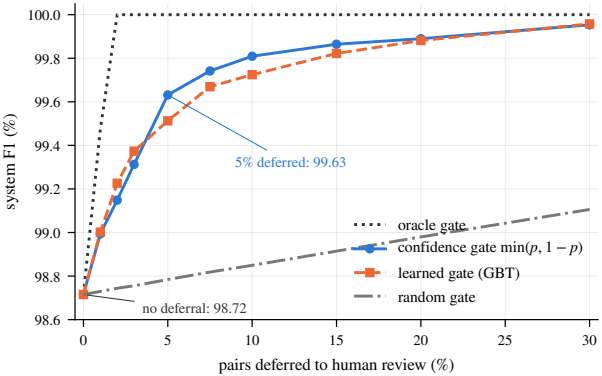
- The best single rule is a token set ratio above 0.99. The production scorer value is the one reported in the benchmark paper.
- The same forest applied to all **375,842 other pairs** of the benchmark gives 98.5 F1 (95% CI 98.4 to 98.5) with the same AUC, so the sample was not lucky.

# How many labels does the matcher need



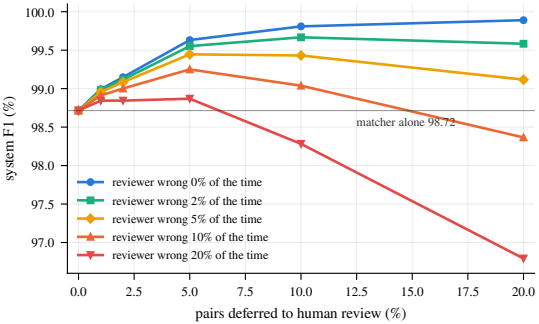

- With 100 labelled pairs the forest already matches the 7B LLM (96.4 vs 95.8). With 1,000 it reaches 97.8, with 10k 98.5, and it saturates near 98.7.
- Logistic regression stays between 96 and 97 at every size, so the gain comes from interactions between features, not from more data.
- A few hundred analyst labels buy more than a 7B model.

# Deferring to a human analyst

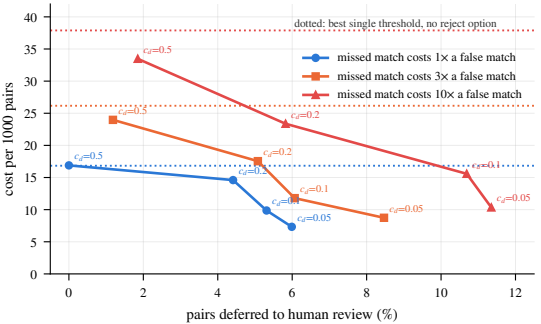



- Reviewing the 5% least confident pairs removes **71% of the remaining errors**. F1 rises from 98.7 to 99.6 and the error count falls from 304 to 87.
- Reviewing 10% removes 85%. A random 5% removes only 5%.
- An oracle gate would need just 2%, so the gap between confidence and oracle is the room left for a better gate.
- The learned gates do not beat plain confidence at any budget. The forest's calibrated uncertainty is already the signal.

# The reviewer need not be perfect and the costs need not be equal

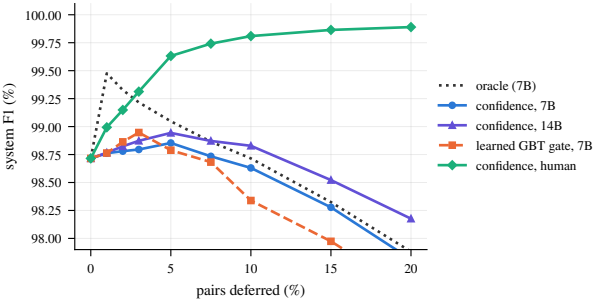



- The matcher is wrong on 24% of the pairs it defers at a 5% budget, so a reviewer who is wrong 10% of the time still lifts F1 to 99.25, and one wrong 20% of the time still helps. Deferral pays as long as the reviewer beats 24% on those pairs.



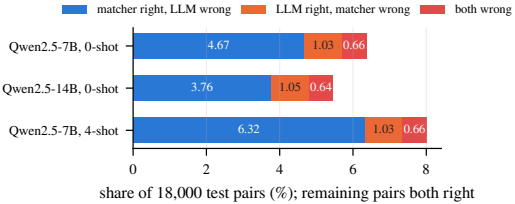

- With a missed match costing 10 times a false match, the Bayes reject region becomes $0.01 < p < 0.9$, 10.7% of pairs are deferred, misses drop from 24 to 3 and the cost per 1000 pairs falls from 37.9 to 15.6.

# Deferring to an LLM




- Handing the least confident 5% to Qwen2.5-7B moves F1 from 98.72 to 98.85. The 14B model reaches 98.94. Past 10% the curves turn down, and at 100% it is the LLM alone at 95.8.
- Four-shot prompting makes the LLM worse, 94.5 alone and no gain as an expert.
- Combining instead of deferring gives the same answer. A tuned weighted average, a logistic stacker and the LLM probability as a 30th forest feature all land at 98.9.
- At any size or prompt the LLM adds **at most 0.2 F1**, against 0.9 for a human at the same budget.

# Why the LLM does not help




**Analyst: different. Matcher 1.00, every LLM: yes**

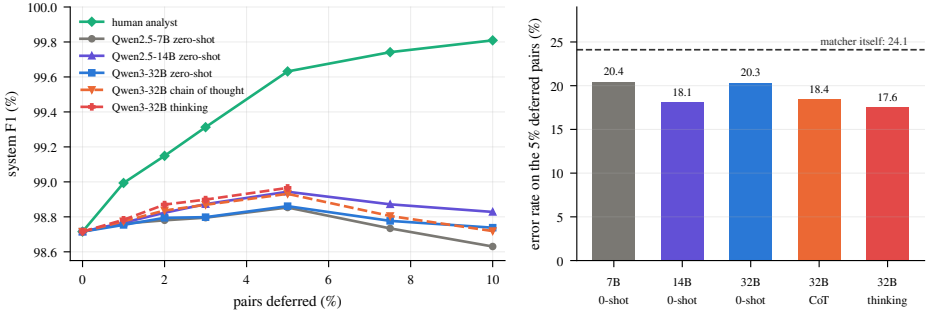
```
A: Vitaliy Anatolyevich Markelov, 1963-08-05, Mordovia
B: Vitaliy Anatolyevich Maarkelov, 1963-08-05, Mordovia
```

**Analyst: different. Matcher 0.97, every LLM: yes**

```
A: Person, [Thai-script name], nationality th, Legislative
Assembly
B: Person, identical name, male, nationality th, same
position
```

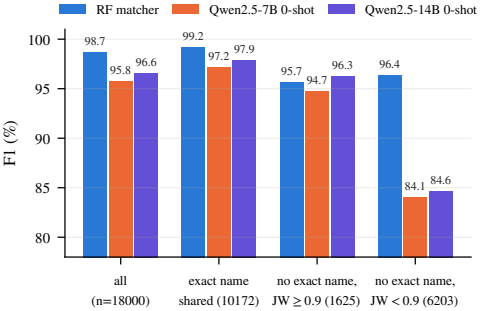
- On the 18k test pairs, 93.6% are right for both systems, 4.7% only the matcher gets right, 1.0% only the LLM, and 0.66% nobody.
- 89 pairs are wrong for the matcher and all three LLM configurations. They share a name but have no dates or identifiers, and the analysts judged them different. 42% of them get $p \geq 0.95$ from the forest, so they are confidently wrong rather than uncertain.
- This information is not in the records, so a bigger model will not recover it. Some are probable label errors. The reject option is the right answer here.

# Does a bigger, reasoning model change this




- Qwen3-32B, four times the parameters, run zero-shot, with a short chain of thought, and in its thinking mode. Alone it is the best LLM at 96.8, still 1.9 below the forest. As the expert on the same 5% it reaches 98.86, 98.93 and 98.97, against 99.63 for the human. **0.25 F1 where the human buys 0.91.**
- The right panel is the reason. On the deferred pairs the forest is wrong 24% of the time and every LLM, 7B to 32B with thinking, is wrong 18 to 20%. Deferral only pays in proportion to how far the expert beats 24%, and none of them gets far.

# Robustness




- On the 7,828 test pairs with no exact shared name the forest keeps 96.1 F1 while the LLMs drop to 88 to 89. Deferring 10% of this hard subset to a human lifts it to 99.5.
- Trained on persons only and tested on organisations the forest gets 90.0, the other direction 97.8, and same-script to cross-script 97.1. The features transfer, but type-specific training matters.
- Five cluster-size count features inflate F1 to 99.0 by leaking how often an entity appears in the data. They are dropped everywhere. Names alone give 97.0, attributes without names 97.4, both together 98.7.
- 53% of test pairs share no entity with training and the forest scores 99.1 F1 on them, so it is not memorising entities.

# Conclusions

- A supervised classical matcher with 29 features reaches **98.7 F1** on OpenSanctions Pairs, above zero-shot 7B and 14B LLMs and far above the published production scorer, with a few hundred labels.
- Treating analyst review as a reject option, the calibrated confidence gate removes **71% of the remaining errors by reviewing 5%** of pairs. Learned gates add nothing, a reviewer with 10% error still helps, and the same rule adapts to asymmetric costs.
- An LLM is a poor expert here. From 7B zero-shot up to a 32B model with reasoning it gains at most 0.25 F1, whether it is deferred to or combined with the matcher, because its error rate on the deferred pairs stays near the matcher's own and the 0.5% hard core of attribute-poor same-name pairs stays hard.
- Two artefacts of the raw benchmark, relational stubs and cluster-size counts, inflate supervised scores and have to be removed.
- Limitations. The LLMs are open models up to 32B, the human is assumed to beat 24% error on the deferred pairs, and the sample's class ratio is not the deployment ratio. Natural next steps are a frontier model on the deferred pairs and deployment-ratio evaluation.

# Backup. Gate quality and calibration

AUC of each gate score on the test split

| gate score | vs matcher error | vs LLM gain |
|---|---|---|
| confidence $\min(p, 1-p)$ | **0.945** | **0.956** |
| GBT error gate | 0.932 | |
| LR error gate | 0.924 | |
| GBT gain gate | | 0.883 |
| LR gain gate | | 0.926 |

- Isotonic calibration is fitted on the 6k validation pairs. The Brier score moves from 0.0140 to 0.0137 and the reliability curve lies on the diagonal.
- The learned gates see the 29 features, the raw and calibrated probabilities, and are trained on 12k training pairs with LLM answers, disjoint from calibration and test data.
- On the pairs deferred at a 10% budget, 84% have a missing date on one side (40% elsewhere), 55% are organisation pairs (33%), and the matcher error rate is 14.4% against 0.3% on the kept pairs.